Assignment - ground vehicles localization - Visual Localization 2026

Overview

Design, implement, and test a **visual localization approach** leveraging the provided data streams from an **Unmanned Ground Vehicle (**UGV) system. The aim is to evaluate your ability to:

- Analyze available data
- Propose a technically localization method using vision
- Prototype or tune part of the system
- Document your reflection, architecture choices, and results

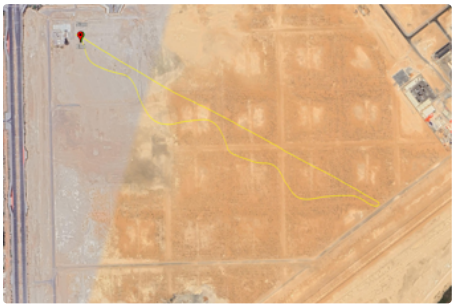
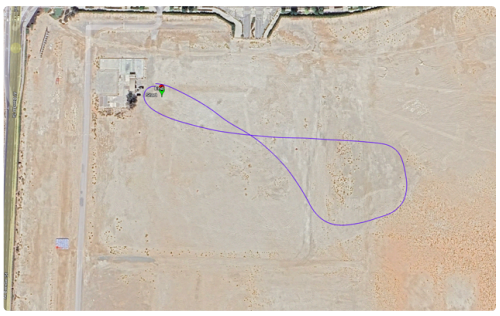
> Note: This assignment focuses on demonstrating your **problem-solving approach** and **engineering judgment** rather than achieving perfect results.

Context

You're joining the **Autonomous UGV Localization team**. A robot “tbuggy” operates in **outdoor environments**. You’re given some ROS bag with about **5 minutes** of driving data that includes:

- A front monocular camera (rectified or raw stream)
- IMU and odometry from the High-grade GNSS/INS unit
- Transform trees for coordinate frame management

Available ROS Bag sequences

Sequence 01	Sequence 02
	

ROS Bag Topics

Link: [visual-localization]

Topic	Type	Description
`/tbuggy/camera_front/camera_info`	`sensor_msgs/CameraInfo`	Camera intrinsics
`/tbuggy/camera_front/image_rect`	`sensor_msgs/Image`	Rectified front camera stream
`/tbuggy/camera_front/image_raw`	`sensor_msgs/Image`	Raw front camera stream
`/tbuggy/imu_ins`	`sensor_msgs/Imu`	IMU data
`/tbuggy/odom`	`nav_msgs/Odometry`	2D Ground truth odometry (x, y, yaw)
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	Transform tree

> Note: As with real-world deployments, the image stream may exhibit occasional frame drops or slight irregularities. These are typical of field-collected data and should be taken into account in your analysis and system design.

Your Tasks

1. Data Exploration

Goal: Get familiar with the dataset and assess its quality.

- Check message frequencies and synchronization
- Inspect camera and IMU signal quality
- Plot the ground truth trajectory to understand the motion

Deliverable: A short summary

---

2. Strategy & Method Selection

Goal: Choose a visual localization approach based on the available data.

- Briefly describe your chosen method (e.g., VO, VIO, hybrid)
- You can use an open-source SLAM/VIO package or build a basic custom pipeline
- Justify your choice based on practicality, robustness, and time constraints

Deliverable: A short write-up explaining your approach and reasoning

> Note: The solution must be viable for **real-time application** (capable of processing data at sensor frame rates). You are **not** constrained by CPU or GPU limitations. Assume the UGV carries a powerful compute unit.

---

3. Implementation

Goal: Build a minimal working setup on one provided bag.

Setup:

- Install and configure your chosen tool or framework
- Create basic launch and config files

Integration:

- Adjust frame IDs and coordinate transforms
- Set initial parameters and input topics
- Log the estimated trajectory in a usable format

Deliverable: A working ROS pipeline with launch/config files
Bonus: A Dockerfile to ensure reproducibility is highly appreciated

> Note: You are **free to convert** these logs to **ROS 2** or any other format if you prefer a different middleware or workflow. If you choose to convert the data, please briefly mention the tool or method used in your report.

---

4. Testing & Analysis

Goal: Evaluate the initial performance and identify strengths/weaknesses.

- Run at least one full bag and record the estimated trajectory
- Compare it to `/tbuggy/odom` as a 2D constraint
- Define Key Performance Indicators (KPIs) for this system
  - Why did you choose these specific metrics ?

  - Report the results of these KPIs
- Identify a few success/failure moments (e.g., where it diverges or holds well)
- Robustness Check (**Optional/Bonus**):
  - Run your system on the second provided rosbag *without* significantly re-tuning parameters.
  - Briefly comment on the performance.

Deliverable: Plots, metrics, and 1–2 paragraphs of commentary

---

5. Documentation

Goal: Summarize your process and results clearly.

- What method did you try, and why?
- What worked well? What didn't?
- What would you improve next?

Deliverable: A concise technical report (PDF or Markdown)

Hints & Tools

- Use `rqt`, `rviz`, `tf_echo`, `image_view`, `plotjuggler`, or any tools you prefer like evo
- Time synchronization can be checked via topic timestamps or rqt_bag
- If you use SLAM packages, adapting them to rectified or raw camera stream and frame IDs is crucial
- Monocular systems inherently lack scale. Since you have an IMU, consider how your VIO algorithm initializes and maintains scale.
- Consider recording output into another bag or CSV for comparison plots
- If you include a `Dockerfile` or setup instructions, it'll make your work easier to test and review.